

Lista de Exercícios de Fixação – Questões Teóricas

1. O que é o Express.js e quais suas principais vantagens?

O Express.js é um framework para Node.js que facilita a criação de servidores web e APIs. Ele fornece uma estrutura simples e flexível para lidar com requisições HTTP.

Principais vantagens:

- Facilidade na criação de rotas
 - Suporte a middlewares
 - Código mais organizado e escalável
 - Grande comunidade e ecossistema
-

2. Quais são os quatro principais tipos de rotas HTTP?

- **GET:** utilizado para obter dados
 - **POST:** utilizado para enviar dados
 - **PUT:** utilizado para atualizar dados
 - **DELETE:** utilizado para remover dados
-

3. Explique o conceito de middleware no Express.js.

Middleware são funções que têm acesso ao objeto de requisição (`req`), resposta (`res`) e à próxima função da aplicação (`next`). Eles são executados durante o ciclo de uma requisição e podem:

- Modificar dados da requisição/resposta
 - Executar validações
 - Tratar erros
 - Controlar autenticação
-

4. Como criar uma conexão básica entre Node.js e MySQL?

Para criar uma conexão básica:

1. Instalar o pacote `mysql2`
 2. Importar o módulo no projeto
 3. Configurar as credenciais (host, usuário, senha, banco)
 4. Criar a conexão utilizando o método de conexão
 5. Executar consultas no banco
-

5. O que é um prepared statement e como ele protege contra SQL Injection?

Prepared Statement é uma forma de executar consultas SQL utilizando parâmetros ao invés de concatenar valores diretamente.

Ele protege contra SQL Injection ao garantir que os dados inseridos pelo usuário sejam tratados como valores, impedindo a execução de comandos maliciosos.

6. Qual é a estrutura básica de um servidor Express.js?

A estrutura básica inclui:

1. Importar o Express
 2. Criar uma instância da aplicação
 3. Definir rotas
 4. Configurar middlewares (opcional)
 5. Iniciar o servidor com `listen`
-

7. Explique a diferença entre rotas globais e rotas específicas.

- **Rotas globais:** são aplicadas a todas as requisições (ex: middlewares globais).
 - **Rotas específicas:** são aplicadas apenas a caminhos ou endpoints específicos.
-

8. Liste três boas práticas ao manipular dados em bancos de dados.

- Utilizar prepared statements
 - Validar e sanitizar dados de entrada
 - Tratar erros adequadamente
 - Utilizar pool de conexões
 - Evitar exposição de dados sensíveis
-

9. Por que é importante usar variáveis de ambiente em aplicações Node.js?

- Protege informações sensíveis
 - Facilita a configuração entre ambientes
 - Evita hardcoding no código
 - Melhora a segurança e organização
-

10. Como tratar erros em consultas SQL dentro do Node.js?

- Utilizar `try/catch` com `async/await`
 - Tratar erros em callbacks ou Promises
 - Registrar logs para análise
 - Retornar mensagens apropriadas para o usuário
-